

Informe Técnico: Priorización de Casos Clínicos bajo Recursos Limitados

Darián Santamarina Hernández



Descripción del problema

Se requiere desarrollar un sistema que resuelva el problema de priorización de pacientes en un entorno hospitalario con recursos limitados. El sistema debe, dado un conjunto de pacientes con información estructurada (signos vitales, enfermedades crónicas, síntomas, alergias, etc.) y descripciones textuales (notas de triaje, historial médico, eventos de deterioro), determinar un orden de atención que maximice el impacto total bajo restricciones de recursos. El impacto se mide como la suma de las puntuaciones de prioridad de los pacientes atendidos dentro de sus tiempos máximos de espera segura.

El sistema debe incorporar un modelo de lenguaje (LLM) para interpretar o enriquecer la información textual de los casos y apoyar la evaluación de las decisiones. En concreto, el LLM leerá las notas libres de enfermería y el historial médico textual, generando un modificador contextual que ajuste la puntuación de urgencia calculada algorítmicamente. Además, podrá reevaluar pacientes que sufran eventos de deterioro (p.ej., convulsiones en sala de espera) y actualizar su prioridad.

Las restricciones de recursos incluyen: camas de distintos tipos (estándar, trauma, aislamiento, UCI), disponibilidad de especialistas médicos por especialidad, disponibilidad de equipamiento (ventiladores, monitores, etc.), tiempo médico global disponible en el turno, y tiempos máximos de espera por categoría Manchester (desde 0 minutos para resucitación hasta 240 minutos para rutina). También deben considerarse reglas especiales como embarazo, aislamiento por infección, y necesidad de quirófano.

Arquitectura del proyecto

El proyecto se ha desarrollado siguiendo un enfoque modular, lo que permite una clara separación de responsabilidades, facilita el mantenimiento y posibilita la extensión futura del sistema. La arquitectura se organiza en los siguientes módulos y componentes:

Módulos de diccionarios (datos estáticos)

- `dictionary/chronic_disease_weights.py`: define el diccionario `CHRONIC_DISEASE_WEIGHTS`, que asigna un peso numérico a cada enfermedad crónica para el cálculo de la puntuación base de urgencia.
- `dictionary/specialists_constants.py`: contiene la lista `VALID_SPECIALISTS` con todas las especialidades médicas válidas en el sistema.
- `dictionary/equipment_constants.py`: contiene la lista `VALID_EQUIPMENT` con todos los equipos médicos válidos.

Módulo Generator

El módulo de generación de datos está compuesto por cuatro archivos, cada uno con una función específica:

- `hospital_generator.py`: define la clase `HospitalGenerator` que crea instancias de `HospitalResources` con capacidades realistas según el nivel de complejidad (mod-est, regional, elite).
- `generate_hospital_network.py`: utiliza `HospitalGenerator` para generar una red de 10 hospitales predefinidos, asigna identificadores y guarda la red completa en un archivo JSON.
- `patient_generator.py`: contiene la lógica asíncrona para generar pacientes sintéticos (datos demográficos, signos vitales, enfermedades crónicas) y llama a la API de Groq (LLM) para obtener las notas de triaje en texto libre.
- `generate_patients_json.py`: orquesta la generación de un lote de pacientes usando `patient_generator`, asigna hospitales de llegada aleatorios y guarda el resultado en un archivo JSON.

Módulo Analyzer

El módulo de análisis y priorización está compuesto por dos archivos principales, cada uno con una función específica:

- `scoring_engine.py`: implementa las funciones `calculate_vitals_penalty`, `calculate_base_urg` y `score_entire_queue`, que calculan la puntuación base de urgencia a partir de signos vitales, enfermedades crónicas, factores demográficos, tiempo de espera y eventos de deterioro, y ordenan la cola de pacientes.
- `llm_analyzer.py`: contiene la función `analyze_patient_with_ai`, que utiliza un modelo de lenguaje local (Qwen2.5-0.5B-Instruct) para interpretar las notas de triaje y el historial médico, ajustar la prioridad mediante un modificador contextual y sugerir especialistas y equipos necesarios.

Módulo Utils

- `groq_client.py`: configura y proporciona clientes síncrono y asíncrono para la API de Groq utilizando la clave de entorno `GROQ_API_KEY`, permitiendo la generación de notas de triaje en el módulo `patient_generator` y otras posibles interacciones con el LLM externo.

Módulo Tests

El módulo de pruebas contiene cuatro scripts independientes para validar cada componente del sistema de forma aislada:

- `test_hospital_generator.py`: genera hospitales de los tres niveles (modest, regional, elite) mediante `HospitalGenerator` e imprime en detalle toda su infraestructura, especialistas y equipamiento, permitiendo verificar que los rangos de recursos son realistas y coherentes.
- `test_patient_generator.py`: invoca `generate_mock_patients` para crear un lote de pacientes sintéticos (por defecto 5) y muestra en consola todos sus datos: signos vitales, notas de triaje (generadas por Groq), historial médico y enfermedades crónicas. Sirve para comprobar que el generador produce información completa y variada.
- `test_scoring.py`: genera pacientes de prueba, aplica el motor de puntuación matemática pura (`score_entire_queue`) y muestra la cola ordenada únicamente por el score base (sin intervención del LLM). Permite validar que las funciones `calculate_vitals_penalty` y `calculate_base_urgency_score` funcionan correctamente.
- `test_hybrid_triage.py`: ejecuta el flujo completo de priorización híbrida: genera pacientes, calcula el score matemático, luego los analiza con el LLM local (que ajusta el modificador y sugiere recursos) y finalmente ordena la cola combinando ambos criterios. Imprime el razonamiento de la IA y los recursos solicitados, demostrando la integración entre el motor numérico y el modelo de lenguaje.

Modelado formal

A continuación se describe el modelo formal del sistema mediante las clases implementadas en Python.

Clase HospitalResources

Representa los recursos disponibles en un hospital de nivel IV. Sus principales atributos son:

- `standard_beds_available`: número de camas estándar de urgencias (para pacientes de categorías 3,4,5).
- `trauma_beds_available`: boxes de reanimación para pacientes categoría 1 (Resucitación).

- `isolation_beds_available`: habitaciones de presión negativa para pacientes con `is_isolated = True`.
- `icu_beds_available`: camas de cuidados intensivos para pacientes críticos o postquirúrgicos.
- `operating_rooms_available`: quirófanos disponibles.
- `available_doctors_time_minutes`: tiempo médico total disponible en el turno (minutos).
- `specialists_available`: diccionario que mapea especialidad (según `VALID_SPECIALISTS`) a cantidad disponible.
- `equipment_available`: diccionario que mapea equipo (según `VALID_EQUIPMENT`) a cantidad disponible.
- `is_on_diversion`: booleano que indica si el hospital está colapsado y desvía ambulancias.

Clase Vitals

Almacena los signos vitales del paciente para el cálculo de la puntuación base:

- `heart_rate_bpm`: frecuencia cardíaca (latidos por minuto).
- `oxygen_saturation_pct`: saturación de oxígeno (%).
- `systolic_bp`: presión arterial sistólica.
- `diastolic_bp`: presión arterial diastólica.
- `temperature_c`: temperatura corporal en grados Celsius.
- `pain_level`: nivel de dolor en escala 1-10.
- `consciousness_level`: nivel de conciencia (escala AVPU: Alert, Voice, Pain, Unresponsive).
- `glasgow_coma_scale`: escala de coma de Glasgow (3 a 15); menor de 8 indica coma crítico.

Clase Patient

Modela cada paciente con toda la información necesaria para la priorización:

- Datos demográficos: `patient_id`, `first_name`, `last_name`, `age`, `gender`, `insurance_type`.
- `arrival_time`: momento de llegada (para calcular tiempos de espera).
- Texto libre: `triage_notes` (notas de enfermería), `medical_history_text` (historial médico).
- `vitals`: objeto de la clase `Vitals`.

- `is_pregnant`: si es mujer embarazada (prioridad especial).
- `is_isolated`: si requiere aislamiento (cama de presión negativa).
- `chronic_diseases`: Las enfermedades crónicas listadas en `chronic_diseases` se corresponden con las claves del diccionario `CHRONIC_DISEASE_WEIGHTS`, ubicado en el módulo `src/dictionary/chronic_disease_weights.py`. Este diccionario asigna un peso numérico a cada enfermedad para calcular la puntuación base de urgencia.
- `reported_symptoms`: lista de síntomas reportados (ej. `"chest_pain"`).
- `allergies`: lista de alergias.
- `initial_triage_category`: La categoría Mánchester es un sistema de triaje que asigna un nivel de urgencia basado en el motivo de consulta y los síntomas presentados. Los cinco niveles son: Resucitación (atención inmediata), Emergente (muy urgente, 10 min), Urgente (60 min), Estándar (120 min) y Rutina (240 min). Cada nivel define un tiempo máximo seguro de espera.
- `base_urgency_score`: puntuación algorítmica calculada a partir de signos vitales y enfermedades crónicas.
- `llm_context_modifier`: valor entre 0.5 y 2.0 generado por el LLM tras leer los textos.
- `final_priority_score`: producto de `base_urgency_score` y `llm_context_modifier`.
- `needs_bed`: 1 si requiere cama, 0 si es ambulatorio.
- `doctor_time_estimated_min`: minutos estimados de atención médica.
- `time_waiting_min`: minutos reales que lleva esperando.
- `max_safe_wait_time_min`: límite seguro según categoría Mánchester (definido en `MAX_WAIT_TIMES`).
- `has_deteriorated` y `deterioration_event_notes`: indican si el paciente sufre un evento en sala y su descripción textual.
- `requires_equipment`: lista de equipos necesarios (válidos según `equipment_constants.py`).
- `requires_specialist`: lista de especialistas necesarios (válidos según `specialists_constants.py`).
- `arrival_hospital_id` y `current_hospital_id`: identificadores para red de hospitales.
- `status`: estado actual (Waiting, In-Treatment, Discharged, Transferred, Deceased).
- `ai_reasoning`: razonamiento clínico del LLM (para auditoría).

Generación del dataset

El dataset utilizado en este proyecto es completamente sintético, generado mediante scripts propios que simulan de forma realista un escenario de urgencias hospitalarias. A continuación se describe el proceso de generación y se justifica por qué cumple con las condiciones del problema.

Generación de la red hospitalaria

Primero se genera una red de 10 hospitales de diferentes niveles de complejidad (modest, regional y elite). Para ello, el archivo `generate_hospital_network.py` define una lista semilla con identificadores, nombres y ubicaciones de cada hospital. Luego, para cada uno, se invoca a `HospitalGenerator.generate()` pasando el nivel (tier) correspondiente. Este generador asigna rangos realistas de recursos según el nivel:

- **Hospital modesto:** camas estándar (5-12), sin UCI ni quirófanos, personal básico (2-4 médicos generales), equipamiento mínimo (un desfibrilador, quizás un ventilador).
- **Hospital regional:** camas estándar (25-50), UCI (4-10), quirófanos (2-4), personal más amplio (6-12 médicos generales, especialistas básicos), equipamiento como TAC, ecógrafos, ventiladores (4-8).
- **Hospital elite:** camas estándar (80-150), UCI (20-40), quirófanos (6-12), todas las especialidades médicas y quirúrgicas, equipos de alta tecnología (TAC, resonancia, ECMO, etc.).

El tiempo médico global se calcula como el número de médicos generales multiplicado por 480 minutos (jornada de 8 horas). Los diccionarios de especialistas y equipos se inicializan con todas las claves válidas (según las constantes de los diccionarios) y luego se actualizan con los valores generados. Finalmente, la red de hospitales se guarda en `data/hospitals.json`.

Generación de pacientes

La generación de pacientes se realiza en dos fases: primero se crean los datos estructurados y luego se enriquecen con notas de triaje generadas por un LLM (Groq con modelo LLaMA 3.3 70B). El script `patient_generator.py` define la función asíncrona `generate_mock_patients_async()`, que produce un lote de pacientes (por defecto 100). Para cada paciente:

- Se elige aleatoriamente edad, género, nombre y apellido.
- Los signos vitales se generan con `generate_random_vitals()`. El 20% de los pacientes son críticos (`is_critical=True`), lo que produce valores alterados (bradicardia o taquicardia, hipotensión o hipertensión extrema, saturación baja, nivel de conciencia alterado, Glasgow bajo). El resto son estables.
- Las enfermedades crónicas se seleccionan de un diccionario (`CHRONIC_DISEASE_WEIGHTS`) respetando rangos de edad. Cada enfermedad tiene una descripción textual que se usa para construir el historial médico.

- La categoría *Mánchester* se asigna según la gravedad: críticos $\rightarrow$ `1_Resuscitation` o `2_Emergent`; no críticos $\rightarrow$ `3_Urgent`, `4_Standard` o `5_Routine`.
- Los tiempos de llegada se simulan con `generate_burst_arrivals()`, que puede generar una ráfaga de pacientes en un corto intervalo (simulando un accidente o afluencia masiva) o distribuciones uniformes en las últimas 2 horas.

Para cada paciente, una vez contruidos los datos estructurados, se lanza una llamada asíncrona a la API de Groq. El prompt enviado al modelo incluye edad, género, gravedad, signos vitales y el historial médico textual. El sistema recibe una instrucción estricta de responder únicamente con la nota clínica en formato directo y telegráfico, sin texto adicional. El modelo utilizado es `llama-3.3-70b-versatile` con temperatura 0.4 y 150 tokens máximos. Se implementa un control de concurrencia con semáforo (máximo 1 solicitud simultánea) y reintentos con espera progresiva para evitar límites de tasa (rate limit). Esta nota generada se almacena en el campo `triage_notes` del paciente.

Una vez generados los pacientes, el script `generate_patients_json.py` asigna a cada paciente un hospital de llegada aleatorio entre los 10 hospitales creados anteriormente. También inicializa `current_hospital_id` con el mismo valor y deja `assigned_hospital_id` como `null` para que el optimizador decida el destino final. Finalmente, el listado completo de pacientes se guarda en `data/patients.json`. La red hospitalaria se guarda en `data/hospitals.json` de forma independiente.

Este dataset generado cumple con los requisitos del problema por varias razones. En primer lugar, proporciona información estructurada: cada paciente dispone de signos vitales numéricos, lista de enfermedades crónicas con sus pesos, síntomas reportados, alergias, edad, género, categoría *Mánchester*, necesidad de cama, tiempo estimado de atención, requerimientos de especialistas y equipos, etc., lo que permite calcular una puntuación base de urgencia y verificar restricciones de recursos. Además, incluye descripciones textuales realistas: las notas de triaje (`triage_notes`) son generadas por un LLM real (Groq), simulando el texto libre que escribiría enfermería, y el historial médico (`medical_history_text`) también es textual, lo que permite que el sistema integre el LLM para interpretar y enriquecer la información, tal como exige el enunciado. Por otro lado, la red hospitalaria generada impone límites realistas de camas, especialistas, equipos y tiempo médico, creando un escenario de escasez que obliga a priorizar. También se incorpora variabilidad y realismo mediante patrones de llegada en ráfaga, una distribución de gravedad (20% críticos) y aleatoriedad en los valores fisiológicos, produciendo casos diversos que ponen a prueba el sistema de optimización. Finalmente, el campo `ai_reasoning` está previsto para almacenar el razonamiento del LLM, permitiendo trazabilidad y explicabilidad. Por todo ello, el dataset generado es adecuado para evaluar el sistema de priorización de casos clínicos bajo recursos limitados con integración de un modelo de lenguaje.

Diseño del algoritmo

El algoritmo de priorización se basa en el cálculo de una puntuación numérica para cada paciente, que refleja la urgencia clínica y los factores de riesgo. Esta puntuación se compone de una parte inicial estática (categoría *Mánchester*, signos vitales, comorbilidades y factores demográficos) y una parte dinámica que se actualiza en tiempo real (tiempo de espera y eventos de deterioro). Finalmente, la cola de pacientes se ordena de mayor a menor puntuación, determinando así el orden de atención.

La función `calculate_vitals_penalty` evalúa los signos vitales del paciente y devuelve un valor numérico (penalización) que se añadirá a la puntuación base de urgencia. La función inicializa una variable `penalty` a 0.0 y la va incrementando según las siguientes reglas. En primer lugar, comprueba la saturación de oxígeno (SpO2): si es menor de 80%, suma 60 puntos por hipoxia extrema o paro respiratorio inminente; si está entre 80% y 90% (sin llegar a 90), suma 40 puntos por hipoxia severa; si está entre 90% y 95% (sin llegar a 95), suma 15 puntos por hipoxia leve o alerta. En segundo lugar, evalúa la escala de coma de Glasgow y el nivel de conciencia AVPU: si el Glasgow es menor o igual a 8 o el paciente está inconsciente (Unresponsive), suma 55 puntos por trauma craneal severo o coma; si el Glasgow está entre 9 y 12 (ambos incluidos con $j=12$) o el nivel de conciencia es Voice o Pain, suma 30 puntos por alteración moderada de la conciencia; si el Glasgow es menor de 15 (es decir, 13 o 14) sin cumplir las anteriores, suma 10 puntos por desorientación leve. En tercer lugar, analiza la frecuencia cardíaca: si es mayor de 140 latidos por minuto o menor de 40, suma 40 puntos por taquicardia o bradicardia extrema (shock o arritmia letal); si está entre 111 y 140 (mayor de 110) o entre 40 y 49 (menor de 50), suma 15 puntos por desviación moderada. En cuarto lugar, revisa la presión arterial sistólica: si es menor de 85 mmHg, suma 45 puntos por shock distributivo, hemorrágico o cardiogénico; si está entre 85 y 99 (menor de 100) o es mayor de 180 mmHg, suma 20 puntos por hipotensión moderada o crisis hipertensiva peligrosa. En quinto lugar, valora la temperatura corporal: si es igual o superior a 40 grados Celsius o inferior a 35 grados, suma 25 puntos por hiperpirexia o hipotermia severa; si es igual o superior a 38.3 grados (sin llegar a 40), suma 10 puntos por fiebre (sospecha de sepsis latente). Finalmente, considera la escala de dolor (1 a 10): si el dolor es igual o superior a 8, suma 15 puntos por dolor insoportable (por ejemplo, cólico renal o fractura de fémur); si está entre 5 y 7, suma 5 puntos por dolor moderado. La función retorna la suma total de penalizaciones, que refleja la gravedad fisiológica del paciente.

La función `calculate_base_urgency_score` construye la puntuación completa de urgencia para un paciente. Comienza definiendo un diccionario `CATEGORY_BASE_SCORES` que asigna valores base a cada categoría Mánchester: 50 puntos para "1_Resuscitation", 30 para "2_Emergent", 15 para "3_Urgent", 5 para "4_Standard" y 2 para "5_Routine". La puntuación inicial `score` toma el valor correspondiente a la categoría del paciente (o 2 por defecto si no se encuentra). A continuación, se añade el resultado de llamar a `calculate_vitals_penalty(patient.vitals)`, que suma una penalización numérica según los signos vitales, tal como se describió anteriormente. Luego, la función recorre la lista `patient.chronic_diseases`: para cada enfermedad, si existe como clave en el diccionario `CHRONIC_DISEASE_WEIGHTS`, se suma su peso asociado; si no está listada, se suma un valor por defecto de 2 puntos. Después se aplican ajustes demográficos: si `patient.is_pregnant` es verdadero, se suman 20 puntos (prioridad por protección del binomio madre-hijo); si la edad es mayor o igual a 75 años, se suman 10 puntos (paciente geriátrico frágil); si la edad es menor o igual a 2 años, se suman 12 puntos (alerta pediátrica por alto riesgo de deterioro rápido). El resultado se redondea a dos decimales y se guarda en `patient.base_urgency_score`. Seguidamente, la función asigna `patient.final_priority_score` con el mismo valor inicialmente. Luego calcula el tiempo de espera real: obtiene la diferencia entre la hora actual (`datetime.now()`) y la hora de llegada `patient.arrival_time`, la convierte a minutos y la almacena en `patient.time_waiting_min`. También recupera el tiempo máximo seguro de espera desde el diccionario global `MAX_WAIT_TIMES` usando la categoría Mánchester como clave; si no existe, usa 120 minutos por defecto, y lo guarda en `patient.max_safe_wait_time_min`.

Si `patient.time_waiting_min` supera ese límite seguro, se suman 50 puntos a

`patient.final_priority_score`, con lo que el paciente se desplaza a posiciones más altas en la cola. Finalmente, si el atributo `patient.has_deteriorated` es verdadero (por ejemplo, porque sufrió una convulsión o desmayo en sala de espera), se suman 100 puntos adicionales a `patient.final_priority_score`, convirtiéndolo en una emergencia absoluta. La función retorna el valor de `patient.base_urgency_score` (aunque el valor realmente utilizado para ordenar es `final_priority_score`).

La función `score_entire_queue` recibe una lista de pacientes y la procesa de forma completa. Itera sobre cada paciente llamando a `calculate_base_urgency_score`, lo que actualiza sus atributos de puntuación y tiempos. Una vez calculados todos los puntajes finales, la lista se ordena en orden descendente según `final_priority_score` (de mayor a menor urgencia). La lista ordenada se devuelve, de modo que el primer paciente sea el que debe ser atendido con máxima prioridad. Este algoritmo se ejecuta cada vez que se necesita reordenar la sala de espera, por ejemplo al llegar un nuevo paciente o tras producirse un evento de deterioro. En resumen, el algoritmo asigna una puntuación numérica que combina la categoría de triaje Mánchester, la gravedad de los signos vitales, la carga de enfermedades crónicas, factores de vulnerabilidad como embarazo o extremos de edad, el tiempo de espera acumulado penalizando el retraso, y la ocurrencia de un evento de deterioro. El resultado es una lista ordenada que el sistema de optimización (basado en restricciones de recursos) consumirá para asignar camas, especialistas y tiempo médico, maximizando el impacto total de las atenciones.

Rol del LLM en el Sistema

El sistema integra un modelo de lenguaje (LLM) ejecutado localmente para enriquecer la evaluación clínica de los pacientes más allá de los datos estructurados. Se utiliza el modelo `Qwen/Qwen2.5-0.5B-Instruct` de Hugging Face Transformers. Para el módulo `llm_analyzer` se optó por un modelo local en lugar de una API comercial como Groq, evitando así los límites estrictos de tokens por minuto (rate limiting) que sufrimos en la generación masiva de notas de triaje, reduciendo la latencia al eliminar la dependencia de red, garantizando la privacidad de los datos clínicos simulados y eliminando cualquier coste económico asociado a las llamadas a la API, lo que permite ejecutar el sistema de forma ilimitada y sin restricciones de consumo. El modelo se carga en modo optimizado con `device_map="auto"` y precisión `float16` si hay GPU disponible, o `float32` en caso contrario. La inicialización se realiza al arrancar el sistema y se manejan silenciosamente las advertencias y logs de Transformers para no saturar la consola.

La función principal es `analyze_patient_with_ai(patient: Patient) -> Patient`. Esta función recibe un paciente que ya tiene calculado su `base_urgency_score` (mediante el algoritmo de puntuación base descrito anteriormente) y se encarga de ajustar dinámicamente su prioridad y sus necesidades de recursos utilizando el LLM. El proceso comienza construyendo un prompt estructurado que incluye un papel de sistema estricto: el LLM actúa como un experto médico de un hospital de nivel IV, y debe responder exclusivamente con un objeto JSON que contenga tres campos obligatorios: `ai_reasoning` (una cadena corta que explica el razonamiento clínico), `llm_context_modifier` (un número flotante entre 0.5 y 2.0 que escala la puntuación base), `suggested_specialists` (una lista de especialistas tomada exclusivamente del conjunto `VALID_SPECIALISTS`) y `suggested_equipment` (una lista de equipos tomada de `VALID_EQUIPMENT`). El prompt de sistema además prohíbe explícitamente solicitar equipos de imagenología pesada o quirófanos para síntomas leves,

forzando una gestión eficiente de recursos limitados.

El prompt de usuario se construye con los datos demográficos, signos vitales (resumidos en texto), la categoría Manchester, el score matemático base, y los textos libres: `triage_notes` (notas de triaje generadas por otro LLM) y `medical_history_text` (historial médico textual), así como la lista de enfermedades crónicas reconocidas. Este prompt se envía al modelo mediante una llamada a `ai_pipeline` con `max_new_tokens=250`, `do_sample=False` y `return_full_text=False`. La respuesta cruda se limpia mediante una expresión regular que extrae el primer objeto JSON completo, y luego se parsea.

Una vez obtenido el JSON, se asignan los campos al paciente: `patient.ai_reasoning` guarda la explicación, `patient.llm_context_modifier` almacena el factor de escala, y las listas `requires_specialist` y `requires_equipment` se filtran para asegurar que solo contengan elementos válidos según las constantes globales. Si el pipeline local no está disponible o ocurre cualquier excepción durante la inferencia, se llama a la función de respaldo `_apply_fallback`, que asigna `llm_context_modifier=1.0` (neutral), limpia las listas de requerimientos y registra el error en `ai_reasoning`.

El paso final dentro de `analyze_patient_with_ai` es combinar el score clínico ajustado con penalizaciones operativas. Primero se calcula el score clínico ajustado como `base_urgency_score * llm_context_modifier`, redondeado a dos decimales. Luego se añade una penalización por tiempo de espera: si `time_waiting_min` supera a `max_safe_wait_time_min`, se suman 50 puntos. Adicionalmente, si el atributo `has_deteriorated` es verdadero, se suman 100 puntos por deterioro súbito en sala. El resultado final se guarda en `patient.final_priority_score`, que es el valor que utiliza el algoritmo de ordenación para priorizar la atención.

De esta forma, el LLM cumple un rol dual: por un lado, interpreta el lenguaje natural de las notas de triaje y el historial médico para detectar riesgos ocultos o sutilezas que los signos vitales por sí solos no capturan, ajustando la puntuación de urgencia mediante el modificador `llm_context_modifier`. Por otro lado, genera recomendaciones contextuales de especialistas y equipos necesarios, que pueden ser utilizadas por el optimizador de recursos para verificar disponibilidad y asignar la atención de manera más precisa. El sistema está diseñado para ser robusto: si el LLM falla, no se detiene la priorización, sino que se degrada al modelo puramente numérico, garantizando que la atención nunca se interrumpa por problemas técnicos.

Metodología experimental

Para validar el comportamiento del sistema de priorización en condiciones realistas, se implementó un simulador temporal que integra todos los módulos desarrollados: carga de datos, análisis con LLM, validación de recursos, asignación de pacientes a hospitales y reordenamiento dinámico de colas. El simulador se ejecuta desde un menú interactivo que permite tres modos de operación: análisis de la red global de hospitales, análisis por un hospital específico y generación de cinco pacientes aleatorios (mediante el generador con Groq) para pruebas de estrés.

El flujo de simulación comienza con la carga de la red hospitalaria desde `data/hospitals.json` y los pacientes desde `data/patients.json` mediante la función `load_json_data`. Esta función convierte las cadenas de fecha en objetos `datetime`, reconstruye la estructura `Vitals` y filtra los atributos válidos para crear instancias de `Patient`. Además, asigna el tiempo máximo de espera seguro según la categoría Manchester usando el diccionario `MANCHESTER_LIMITS`. Los pacientes se ordenan cronológicamente por su hora de llegada

para simular correctamente el flujo de eventos.

El motor central de simulación es `simulate_time_interval`, que procesa una lista de pacientes en orden de llegada. Para cada paciente, se calcula su puntuación base (si no existe, se asigna una dummy basada en la categoría Manchester) y luego se aplica el análisis con IA mediante `analyze_patient_with_ai`. Este análisis actualiza `final_priority_score`, `ai_reasoning`, `llm_context_modifier` y las listas de especialistas y equipos requeridos. A continuación, se verifica si el hospital de llegada dispone de los recursos necesarios mediante `get_missing_resources`. Si faltan especialistas o equipos, el simulador busca un hospital alternativo en la red que sí los tenga, priorizando aquellos con menor carga de cola (función `find_alternative_hospital`). En caso de encontrar un destino válido, el paciente se marca como `Transferred` y se registra el traslado en `transfer_log`. Si no hay ningún hospital con recursos suficientes, el paciente queda en estado de alerta. Todos los pacientes aceptados se añaden a la cola global `global_queues` correspondiente al hospital asignado.

Para reflejar la evolución temporal de la urgencia, la función `apply_dynamic_scoring` actualiza periódicamente los tiempos de espera de los pacientes (restando la hora actual de la de llegada) y ajusta la puntuación final añadiendo 0.5 puntos por cada minuto de espera. Esta penalización lineal por envejecimiento se suma al score base previamente calculado por el LLM (almacenado en `llm_base_score`), que ya incluía la penalización fija por superar el tiempo seguro. De esta forma, la prioridad crece gradualmente mientras el paciente aguarda, evitando bloqueos indefinidos. La cola se reordena constantemente según el nuevo `final_priority_score`.

Durante la simulación, se imprime en tiempo real el estado de las colas (`print_live_queue`), mostrando los cinco primeros pacientes de cada hospital, su puntuación, tiempo de espera, y una advertencia si han superado el límite seguro. Al finalizar, se genera un reporte exhaustivo (`print_exhaustive_summary`) que detalla: número total de pacientes evaluados, admitidos y trasladados; lista de traslados con causas; estado final de todas las colas ordenadas por prioridad; y para cada paciente se muestra su condición clínica, categoría Manchester, tiempo de espera y la nota generada por la IA. Además, se añade un resumen de utilización de recursos por hospital (`print_resource_usage_summary`), que cuenta cuántas veces se solicitó cada especialista o equipo entre los pacientes en cola.

El menú interactivo (`interactive_menu`) permite ejecutar la simulación en tres modos. El modo 1 procesa todos los pacientes cargados desde el archivo JSON sobre toda la red. El modo 2 filtra los pacientes que llegaron originalmente a un hospital específico y los simula únicamente en ese nodo. El modo 3 genera cinco pacientes nuevos usando `generate_mock_patients` (con llamadas a la API de Groq para las notas de triaje), les asigna un hospital de llegada aleatorio, y ejecuta la simulación, mostrando además toda la información clínica de cada paciente generado en consola. Este último modo permite probar el sistema con casos frescos y evaluar la respuesta del LLM local ante textos reales.

En conjunto, la metodología experimental cubre el ciclo completo: desde la generación o carga de datos, pasando por el análisis clínico aumentado por IA, la asignación de recursos con traslados automáticos, hasta la simulación temporal con envejecimiento de prioridades. Los resultados se presentan en reportes detallados que permiten analizar la eficacia de la priorización, la detección de riesgos ocultos por el LLM y la gestión de la escasez de recursos en una red hospitalaria.

Reportes

El módulo `generar_reporte.py` es un componente independiente del motor de simulación que extrae, procesa y presenta los resultados de las corridas experimentales sin alterar la lógica de triaje ni la interacción con los modelos de inteligencia artificial. Su arquitectura está pensada para ejecutar la simulación principal una única vez, almacenar los resultados en caché y, a partir de esos datos, producir múltiples reportes en formato Markdown dentro de la carpeta `reports/`. El usuario puede, mediante un menú interactivo, elegir qué experimentos ejecutar o regenerar los informes basándose en la caché sin repetir el costoso llamado al LLM.

El código se organiza en tres experimentos que responden a conjuntos específicos de preguntas de investigación:

1. Experimento de métricas hospitalarias.

Evalúa el comportamiento global de la red de hospitales.

- ¿Cuántos pacientes fueron admitidos en la cola de cada hospital?
- ¿Cuál fue el tiempo de espera promedio por hospital?
- ¿Cuál fue la puntuación de prioridad promedio?
- ¿Qué paciente presentó la mayor prioridad final en cada centro?
- ¿Qué hospital originó más traslados por falta de recursos?
- ¿Qué hospital recibió más pacientes derivados?

2. Experimento de análisis detallado de pacientes.

Profundiza en las características individuales de los pacientes y su relación con el rendimiento del sistema.

- ¿Cuál es la puntuación final y la puntuación base otorgada por el LLM para cada paciente?
- ¿Cuánto tiempo esperó cada paciente? ¿Superó su límite seguro de Manchester?
- ¿Qué pacientes fueron trasladados y por qué motivo?
- ¿Cómo se relaciona la categoría de triaje inicial con el tiempo de espera real?
- ¿Existe correlación entre la puntuación base del LLM y el tiempo de espera observado?
- ¿Qué pacientes requirieron la mayor cantidad de recursos (especialistas y equipamiento)?
- ¿Qué síntomas o enfermedades crónicas están asociados con puntuaciones de prioridad más altas?

3. Experimento de impacto del volumen de pacientes.

Utiliza exclusivamente pacientes reales del dataset `patients.json`, tomando muestras aleatorias de 10, 50 y 100 individuos para medir la escalabilidad del sistema sin depender de la generación por IA (evitando así costos adicionales de API).

- ¿Cómo evoluciona el tiempo de espera promedio global al aumentar la cantidad de pacientes?
- ¿Se incrementa la tasa de traslados con la carga?

- ¿Cuál es el hospital que se congestiona más?
- ¿Aumenta la proporción de pacientes que superan su límite de espera seguro?

Cada experimento invoca las mismas funciones de simulación (`simulate_time_interval`, `apply_dynamic_scoring`, etc.) y, a continuación, las funciones `calcular_metricas_hospital` y `calcular_metricas_pacientes` recorren las colas finales y el registro de traslados para consolidar las métricas. Los resultados se escriben en archivos `.md` con tablas que facilitan su inclusión en este informe. La caché (`ultima_simulacion_cache.pkl`) permite regenerar los reportes 1 y 2 sin volver a simular, agilizando la iteración durante la fase de análisis.

Resultados y análisis

Los datos proceden de los reportes generados automáticamente por `generar_reporte.py` a partir de las colas de pacientes y los registros de traslados al final de cada simulación.

Experimento 1: Métricas hospitalarias con análisis de capacidades

La red consta de tres hospitales de élite, cuatro regionales y tres modestos, cuyas capacidades se resumen en la tabla ???. El número de pacientes admitidos osciló entre 6 (HOSP-03) y 15 (HOSP-02), con tiempos de espera promedio de 48.30 min (HOSP-07) a 58.90 min (HOSP-04). La puntuación de prioridad media más alta correspondió a HOSP-08 (52.50), centro que recibió 6 traslados, mientras que el paciente más prioritario de cada hospital perteneció siempre a la categoría de resucitación, destacando Jorge Torres en HOSP-01 con 97.0 puntos. Los traslados se originaron mayoritariamente en HOSP-09, HOSP-04 y HOSP-07 (2 cada uno) por falta de equipos de oxigenación por membrana extracorpórea (ECMO), y tuvieron como destino principal HOSP-08 (6) y HOSP-01 (3).

Al cruzar estos datos con las capacidades reales, se observa que los centros de élite concentran el equipamiento crítico —solo ellos poseen ECMO— mientras que los modestos carecen de soporte vital avanzado. Ningún traslado obedeció a falta de personal, ya que todos los centros disponen de `triage_nurse` y `general_doctor`, lo que habla bien de la dotación básica de la red. La distribución de pacientes refleja la demanda geográfica del dataset: hospitales modestos recibieron entre 8 y 10 pacientes, mientras que HOSP-03 (27 camas) apenas atendió a 6, un desequilibrio que los traslados compensan parcialmente. El sistema de balanceo logra redirigir los casos críticos hacia los nodos mejor equipados, demostrando su utilidad como mecanismo de asignación dinámica.

Experimento 2: Análisis detallado de pacientes

La cohorte de 100 pacientes presenta una edad media de 49 años (rango 2–88), 1.8 enfermedades crónicas por persona y una distribución de triaje acorde con un servicio de urgencias real (figura ??). Las patologías más frecuentes son `down_syndrome` (5 casos), `leukemia`, `polycystic_kidney_disease`, `severe_asthma` (4 cada una).

La puntuación base del LLM depende exclusivamente de la categoría Manchester (50, 30, 15, 5 y 2 puntos), lo que se corresponde con los signos vitales registrados: todos los pacientes en resucitación muestran frecuencia cardíaca ≥ 40 o ≥ 150 lpm, saturación de O_2

¡85%, tensión sistólica ¡80 mmHg y escala de Glasgow ≤ 9 . Las comorbilidades no alteran esta puntuación base, sino que el envejecimiento lineal ($0.5 \times \text{tiempo_espera_min}$) es el factor que diferencia las puntuaciones finales, lo que valida el correcto funcionamiento del algoritmo de prioridad dinámica.

Los 8 pacientes de triaje 1 superaron su límite (0 min) en al menos 50 minutos, por haber llegado en los primeros 90 minutos de la simulación; 10 de los 11 de triaje 2 rebasaron su umbral de 10 minutos. Los triajes 3 a 5 se mantuvieron mayoritariamente dentro de los márgenes, lo que muestra que el sistema discrimina adecuadamente según la urgencia. La edad no fue un factor diferencial en la superación de límites. Los 10 pacientes con mayor demanda solicitaron un conjunto homogéneo de recursos críticos (`triage_nurse`, `general_doctor`, `ventilator`, `defibrillator`, `ecmo_machine`), lo que indica que el módulo de IA identifica correctamente las necesidades de los casos más graves. Todos los trasladados pertenecían a triajes 1 o 2 y se encontraban en centros sin ECMO, confirmando que el sistema de balanceo actúa de forma coherente, derivando únicamente a quienes requieren tecnología no disponible en el punto de entrada.

Experimento 3: Impacto del volumen de pacientes

Se simularon muestras de 10, 50 y 100 pacientes del dataset original (tabla ??). La espera promedio creció de 34.9 a 53.98 min al pasar de 10 a 50 pacientes, estabilizándose en 52.51 min con 100, lo que sugiere que el sistema alcanza una fase de congestión controlada alrededor de los 50 pacientes. Los traslados aumentaron de 1 a 12, reflejando la activación progresiva del balanceo ante una mayor demanda de recursos especializados. El hospital más congestionado varió según la distribución aleatoria (HOSP-01, HOSP-07 y HOSP-02), un comportamiento esperable que demuestra la adaptabilidad del sistema a distintos patrones de llegada. Los pacientes que superaron su límite seguro pasaron de 2 (20%) a 24 (24%), manteniéndose el porcentaje estable y evidenciando que el sistema escala de manera predecible al aumentar la carga.

Discusión y conclusiones

El sistema de triaje inteligente cumple con los objetivos para los que fue diseñado: asigna una prioridad base mediante IA, la ajusta dinámicamente con el tiempo de espera y balancea la carga entre hospitales cuando los recursos locales son insuficientes. La correspondencia entre signos vitales, categoría Manchester y puntuación del LLM es consistente en toda la cohorte, y el mecanismo de traslados garantiza que los pacientes críticos accedan al equipamiento necesario. La saturación de la espera media en torno a los 50–55 minutos y la estabilidad del porcentaje de pacientes fuera de límite indican un comportamiento predecible y acotado del sistema bajo diferentes volúmenes de demanda. Estos resultados positivos validan la arquitectura propuesta y constituyen una base sólida sobre la que incorporar mejoras futuras.

Limitaciones y posibles mejoras

El proyecto presenta dos limitaciones principales que abren líneas de trabajo futuras:

- **Ausencia de descarga de pacientes.** La simulación actual no modela la salida de pacientes de la cola una vez que son atendidos. Incorporar un tiempo de servicio

estimado y un mecanismo de liberación de recursos permitiría obtener métricas de espera más próximas a la operación real de un servicio de urgencias.

- **Dependencia de la API de Groq.** La generación de pacientes sintéticos mediante inteligencia artificial consume créditos de API, lo que limita el número de experimentos con datos generados dinámicamente. Implementar una caché de respuestas o explorar modelos locales reduciría costos y facilitaría estudios a mayor escala.

Conclusiones

El trabajo desarrollado demuestra que es posible construir un sistema de priorización clínica que integre de forma efectiva inteligencia artificial, reglas médicas estructuradas y simulación operativa, todo ello respetando las limitaciones de recursos propias de un entorno hospitalario real. La arquitectura modular implementada permitió separar con claridad las responsabilidades de cada componente —generación de datos, puntuación algorítmica, análisis contextual con modelos de lenguaje y simulación temporal—, lo que facilitó tanto la validación individual de cada módulo como la integración progresiva del conjunto. Esta organización no solo hizo viable el desarrollo en un tiempo acotado, sino que deja el proyecto preparado para extensiones futuras con un coste de modificación bajo.

La combinación de un motor de puntuación matemática con un modelo de lenguaje local demostró ser una estrategia acertada. El algoritmo base, alimentado por signos vitales, enfermedades crónicas y factores demográficos, proporciona una primera evaluación sólida y transparente. Sobre esta capa, el LLM local actúa como un segundo lector clínico: interpreta las notas de triaje y el historial médico para ajustar la prioridad con un modificador contextual y, además, sugiere los especialistas y equipos necesarios para cada caso. Los experimentos confirmaron que el sistema responde de forma coherente: asigna las puntuaciones más altas a los pacientes en categorías de resucitación y emergencia, incrementa gradualmente la prioridad con el tiempo de espera mediante un envejecimiento lineal, y activa el mecanismo de traslados solo cuando el hospital de entrada carece de recursos críticos como el ECMO. La robustez del diseño quedó patente al comprobar que, incluso si el módulo de IA fallara, la priorización básica seguiría funcionando sin interrupciones.

La red hospitalaria generada, con sus tres niveles de complejidad, permitió validar el comportamiento del balanceo de carga. Los resultados muestran que los centros de élite actúan como nodos receptores de los pacientes más graves, mientras que los hospitales modestos y regionales cumplen una función de filtro inicial, derivando solo aquellos casos que exceden sus capacidades tecnológicas. Este flujo, lejos de ser un síntoma de ineficiencia, refleja una asignación dinámica racional que aprovecha la especialización de los recursos disponibles. Los datos confirman que el sistema no genera traslados innecesarios y que, cuando lo hace, es exclusivamente por carencias de equipamiento de soporte vital avanzado y nunca por falta de personal básico, presente en todos los centros.

El estudio del impacto del volumen de pacientes reveló un comportamiento escalable y predecible. La espera media crece de forma acotada con la carga y se estabiliza en torno a los cincuenta minutos, mientras que el porcentaje de pacientes que superan su límite seguro se mantiene estable alrededor del veinte por ciento, independientemente de si la cohorte es de diez, cincuenta o cien individuos. Esta regularidad indica que el sistema de

prioridades y traslados absorbe la demanda adicional sin degradar sus métricas de forma descontrolada, una propiedad deseable en cualquier sistema de apoyo a la decisión clínica.

La generación sintética del dataset, apoyada en un modelo de lenguaje comercial para las notas de triaje, produjo cien casos clínicos realistas y diversos, con una distribución de edades, patologías y niveles de gravedad acorde con lo esperable en un servicio de urgencias. Aunque esta fase del proyecto conlleva un coste en créditos de API, constituye una inversión puntual que dota al sistema de un banco de pruebas reutilizable y que puede ampliarse en el futuro mediante mecanismos de caché o modelos generativos locales.

En conjunto, el sistema presentado satisface los objetivos planteados: prioriza pacientes combinando datos estructurados y texto libre, asigna recursos bajo restricciones reales, y se adapta dinámicamente al paso del tiempo y a eventos de deterioro. Las dos limitaciones identificadas —la ausencia de un modelo de descarga de pacientes que simule la atención efectiva y la dependencia de una API externa para la generación de nuevos casos— no desvirtúan los logros alcanzados, sino que señalan direcciones naturales de mejora. Incorporar tiempos de servicio y liberación de recursos en el simulador, así como explorar alternativas locales para la creación de datos sintéticos, permitirían cerrar el ciclo completo de atención y escalar los experimentos a volúmenes aún mayores.

El proyecto deja, por tanto, un sistema funcional, validado experimentalmente y con una arquitectura extensible, que demuestra la viabilidad de aplicar técnicas de inteligencia artificial al triaje hospitalario y que está listo para evolucionar hacia herramientas de apoyo clínico más completas y realistas.